Reviewer Pack — Minimal Vertex Separability of Regular Graphs and Resolution…

Entry 4669bcbdcd15 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 16:15:11 UTC

Minimal Vertex Separability of Regular Graphs and Resolution Proofs

Entry ID: `4669bcbdcd15` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-21 16:15:11 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry **Field B** (complexity object): Resolution Proof Complexity

Statement:

> {'sentence_1': 'For any regular graph G with girth ≥ 5, the Tseitin formula on G has a resolution proof of length at least 2^g(G), where g(G) is the minimal vertex separation of G.', 'sentence_2': 'The vertex separation of a regular graph is poly-time computable and upper-bounded by O(n).', 'sentence_3': 'For non-regular graphs or graphs with girth < 5, the resolution proof length is at most 2^(n/4).'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Combinatorial geometry provides tools to study the geometric structure of graphs, which may reveal invariants that are hard to simulate on general graphs.', 'sentence_2': 'Regular graphs have well-defined properties such as girth and vertex separation, which could be exploited for proving lower bounds on complexity classes.', 'sentence_3': 'Minimal vertex separation might capture the inherent difficulty of expressing certain graph properties in resolution proofs.'}

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `6a8198c2965b66ce`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For regular graphs G with girth ≥ 5, if the Tseitin formula on G has a resolution proof length of at least 2^g(G), then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - minimal vertex separation regular graph AND resolution proof complexity - Tseitin formula AND minimal vertex separation AND resolution proof length - combinatorial geometry AND girth ≥ 5 AND resolution proof complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def generate_regular_graph(n, degree):
    if (n * degree) % 2 != 0:
        return None
    graph = {i: [] for i in range(n)}
    edges_used = set()
    for v in range(n):
        for u in range(v + 1, n):
            if len(graph[v]) == degree and len(graph[u]) == degree:
                continue
            if (v, u) not in edges_used and (u, v) not in edges_used:
                graph[v].append(u)
                graph[u].append(v)
                edges_used.add((v, u))
    return graph

def tseitin_formula(graph):
    n = len(graph)
    literals = {i: f'x{i}' for i in range(n)}
    clauses = []
    for v in range(n):
        clause = [literals[v]]
        for u in graph[v]:
            clause.append(-literals[u])
        clauses.append(clause)
    return clauses

def resolution_proof_length(clauses):
```

```
    def resolve(clause1, clause2):
        new_clause = []
        for literal in clause1:
            if -literal in clause2:
                continue
            new_clause.append(literal)
        return new_clause

    def is_tautology(clause):
        pos_literals = {abs(l) for l in clause}
        neg_literals = {-l for l in clause}
        return pos_literals & neg_literals

    queue = clauses.copy()
    while True:
        found_new_clause = False
        for i in range(len(queue)):
            for j in range(i + 1, len(queue)):
                new_clause = resolve(queue[i], queue[j])
                if not new_clause or is_tautology(new_clause):
                    continue
                if new_clause not in queue:
                    queue.append(new_clause)
                    found_new_clause = True
        if not found_new_clause:
            break
    return len(queue)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []
    for n in n_values:
        graph = generate_regular_graph(n, degree=3)
        if not graph:
            continue
        clauses = tseitin_formula(graph)
        proof_length = resolution_proof_length(clauses)
        g_G = len(graph) - max(len(neighbors) for neighbors in graph.values())
        results.append({
            "n": n,
            "g_G": g_G,
            "proof_length": proof_length
        })
    if not results:
        return {
            "metric_name": "resolution_proof_length",
            "metric_value": 0.0,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }
    total_length = sum(result["proof_length"] for result in results)
    avg_length = total_length / len(results)
    conjecture_holds = all(length >= 2**g_G for result in results for length, g_G in zip([result['
    return {
```

```
        "metric_name": "resolution_proof_length",
        "metric_value": avg_length,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 8)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

    avg_length = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={avg_length} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_length} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce306feb.py", line 113, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce306feb.py", line 81, in run_trial
    clauses = tseitin_formula(graph)
              ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce306feb.py", line 40, in tseitin_formu
    clause.append(-literals[u])
                  ^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means we cannot verify if the resolution proof length meets the conjectured lower bound of at least 2^g(G). | next: Investigate and fix the error in the test code to ensure it can run to completion and provide the necessary data for verification.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14794
2	preregistration	ollama_remote	glm4:latest	0	0	8856
3	novelty	ollama_remote	glm4:latest	0	0	7990
4	novelty	ollama_remote	glm4:latest	0	0	8864
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19278
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13438
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12924
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11809
9	judge	ollama_remote	glm4:latest	0	0	11783

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 109738 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/4669bcbdcd15.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4669bcbdcd15.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/4669bcbdcd15.tar.gz` (if generated)